

From Pixels to Physics: Imitation and Adaptation of Human Motions from Casual Videos

*submitted in partial fulfillment of the requirements
for the degree of*

MASTER OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

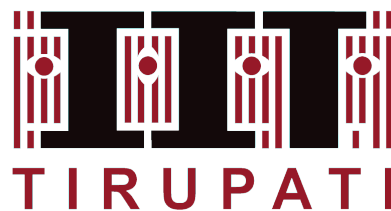
by

LOGESH .V CS24M106

Supervisor(s)

Chalavadi Vishnu

भारतीय प्रौद्योगिकी संस्थान तिरुपति



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY TIRUPATI**

October 30, 2025

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission to the best of my knowledge. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Place: Tirupati
Date: October 30, 2025

Signature
Logesh .V
CS24M106

BONA FIDE CERTIFICATE

This is to certify that the report titled **From Pixels to Physics: Imitation and Adaptation of Human Motions from Casual Videos**, submitted by **Author(s)**, to the Indian Institute of Technology, Tirupati, for the award of the degree of **Master of Technology**, is a bona fide record of the project work done by him under my supervision. The contents of this report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Place: Tirupati
Date: October 30, 2025

Chalavadi Vishnu
Guide
Assistant Professor
Department of Computer
Science & Engineering
IIT Tirupati - 517619

Literature Survey

Chapters 1-8

This collection of research papers addresses the fundamental challenge of generating realistic human skeleton animations from motion capture (mocap) data. Despite varying methodological approaches, several common themes emerge across these works. First, there is a consistent focus on **deep generative models**, with multiple papers employing Generative Adversarial Networks (GANs) and their variants (Papers 1, 2, 3, 4, 6), while more recent works explore **diffusion models** (Papers 7, 8) in policy spaces. Second, there is significant emphasis on **temporal modeling**, with architectures ranging from convolutional sequence models (Paper 1) to transformers (Paper 6). Third, many papers grapple with **physical plausibility**, either through explicit physics-based constraints (Papers 2, 7, 8) or implicit learning of physical realism via discriminators. Fourth, **controllable generation** emerges as a key requirement, addressed through action conditioning (Papers 1, 6), task control (Paper 5), and interactive control mechanisms (Papers 2, 3). Finally, the modeling of **skeletal structure** remains central, with several works employing graph convolutional networks (Papers 4, 6) to explicitly capture kinematic relationships between joints.

Chapters 9-12

This collection of papers addresses the challenging problem of inferring 3D human motion capture data from 2D video inputs. Despite their different technical approaches, these works share several fundamental challenges and methodological commonalities. First, they all confront the inherent **2D-to-3D ambiguity problem**, requiring sophisticated methods to lift monocular or sparse multi-view video evidence into accurate 3D skeletal motions (Papers 9, 10, 11, 12). Second, there is consistent emphasis on **temporal coherence and dynamics modeling**, with Papers 9 and 10 explicitly focusing on learning human dynamics from video sequences, while Papers 11 and 12 extend this to longer-term motion generation. Third, these works demonstrate evolving approaches to handling **viewpoint invariance and novel view synthesis**, with Paper 11 specifically addressing sparse multi-view scenarios and Paper 12 (ViMo) representing the state-of-the-art in extracting motions from casual, unconstrained videos. Finally, all methods must bridge the **domain gap** between image-based features and

3D kinematic representations, employing various deep learning architectures to translate visual patterns into geometrically accurate skeletal motions while maintaining physical plausibility across generated sequences.

Proposed Methodology and Plan of Action

This research proposes a comprehensive pipeline for generating physically plausible and adaptive character motions directly from casual video inputs. The methodology is structured as a three-stage framework that progressively bridges the gap between visual perception and physics-based simulation, with a novel contribution addressing critical adaptation challenges.

Stage 1: Visual Motion Acquisition

The initial phase focuses on extracting raw human motion data from unconstrained video footage. Building upon recent advances in video-based motion capture, this stage will employ a vision-based model to reconstruct 3D skeletal sequences from monocular or sparse multi-view video inputs. The primary objective is to obtain temporally coherent kinematic motions that preserve the stylistic qualities and motion characteristics observed in the source videos, serving as the foundational input for subsequent physics-based processing.

Stage 2: Physics-Based Motion Imitation

The extracted kinematic motions are then translated into physically plausible character control policies. This stage involves developing a physics-based imitation learning framework that enables simulated characters to reproduce the reference motions while maintaining dynamic balance and responding to environmental constraints. The approach will incorporate composite motion learning capabilities to handle complex, multi-task behaviors, ensuring that the generated motions not only mimic the visual appearance but also adhere to physical laws through continuous interaction with a physics simulation environment.

Stage 3: Adaptive Policy Refinement

A critical challenge emerges at the interface between vision-derived motions and physics simulation: kinematic inaccuracies from the vision system and simulation-environment discrepancies often lead to unstable or suboptimal behaviors when deploying the learned policies. To address this fundamental limitation, we introduce a novel policy adaptation mechanism that autonomously refines and adjusts the control policies during deployment.

This adaptive component operates by continuously monitoring the performance gap between expected and actual motion execution, identifying systematic errors and dynamic mismatches that were not apparent during training. Through online policy refinement, the system learns to compensate for these discrepancies without requiring additional expert demonstrations or manual tuning. This represents a significant advancement over conventional pipelines by enabling robust transfer of vision-based motions to physics simulation under varying conditions and inherent estimation errors.

The integrated pipeline thus progresses from visual perception to physical embodiment, culminating in an adaptive control system that ensures robust performance despite the uncertainties introduced at each stage of the processing chain.

Contents

Abstract: Literature Survey	i
Preface: Proposed Plan of Action	ii
1 Convolutional Sequence Generation for Skeleton-Based Action Synthesis (2019)	2
1 Datasets Used	2
2 Methodology	2
2.1 Latent Vector Sequence Generation	2
2.2 Graph Upsampling and Pyramid	3
2.3 Generative Adversarial Training	3
2 A GAN-Like Approach for Physics-Based Imitation Learning and Interactive Character Control (2021)	5
1 Dataset Used	5
2 Methodology	5
2.1 Key Improvements Over GAIL	5
2.2 Policy Training	6
3 GAN-based Reactive Motion Synthesis with Class-aware Discriminators for Human–Human Interaction (2021)	8
1 Datasets Used	8
2 Methodology	8
2.1 Generator: Part-based Attentive Recurrent Network	9
2.2 Discriminator: Class-aware Dual Classifier	9
2.3 Loss Functions	10
4 Generative Adversarial Graph Convolutional Networks for Human Action Synthesis (2022)	11
1 Datasets Used	11
2 Methodology	11
2.1 Graph Convolutional Formulation	11

2.2	Conditional Adversarial Training	12
5	Composite Motion Learning with Task Control (2023)	14
1	Dataset Used	14
2	Methodology	14
2.1	Core Components	14
2.2	Theoretical Backing	15
6	ActFormer: A GAN-based Transformer towards General Action-Conditioned 3D Human Motion Generation (2023)	16
1	Datasets Used	16
2	Methodology	16
2.1	Single-Person Motion Generation	17
2.2	Multi-Person Motion Generation	17
2.3	Generative Adversarial Training	18
7	Policy-Space Diffusion for Physics-Based Character Animation (2025)	19
1	Dataset Used	19
2	Methodology	19
2.1	Mathematical Formulation	19
8	Policy-space Interpolation for Physics-based Characters (2025)	22
1	Introduction	22
2	Dataset Used	22
3	Methodology	22
3.1	Network Architecture and Training	24
9	Learning 3D Human Dynamics from Video (2019)	25
1	Datasets Used	25
2	Methodology	25
2.1	Problem Formulation	26
2.2	3D Human Mesh Representation	26
2.3	Temporal Encoder and 3D Regression	26
2.4	Loss Functions	27
2.5	Training Strategy	27

10 Predicting 3D Human Dynamics from Video (2019)	28
1 Datasets Used	28
2 Methodology	28
2.1 3D Mesh Representation	28
2.2 Causal Temporal Encoder	29
2.3 Autoregressive Prediction	29
2.4 Loss Functions	29
11 Novel View Synthesis of Human Interactions from Sparse Multi-view Videos (2022)	31
1 Datasets Used	31
2 Methodology	31
2.1 Multi-entity Motion Capture	31
2.2 Layered Neural Scene Representation	32
2.3 Network Training	32
12 ViMo: Generating Motions from Casual Videos (2024)	34
1 Datasets Used	34
1.1 Primary Training Dataset	34
1.2 Generated Dataset	34
1.3 Other Referenced Datasets	34
2 Methodology	35
2.1 Technical Framework Overview	35
2.2 Diffusion Model Foundation	36
2.3 Network Architecture Details	37
2.4 Motion Representation Details	38
2.5 Loss Functions Formulation	38
2.6 Training and Sampling Procedure	39
2.7 Advanced Applications Implementation	40
2.8 Key Technical Innovations	41
2.9 Applications	42

Paper 1

Convolutional Sequence Generation for Skeleton-Based Action Synthesis (2019)

Authors: Sijie Yan, Zhizhong Li, Yuanjun Xiong, Huahan Yan, Dahua Lin

1 Datasets Used

- **NTU-RGB+D [?]**: A real-world action dataset with 56,000 short clips from 60 action categories, captured by Kinect sensors. Each skeleton has 25 joints.
- **MikuDance**: A new dataset collected by the authors, containing 201 long dancing sequences (3–5 minutes each), totaling 10 hours and 1 million frames. Each skeleton has 46 joints, including finger joints, with rotational information in quaternions.

2 Methodology

The paper proposes a **Convolutional Sequence Generation Network (CSGN)** for synthesizing long-duration skeleton-based action sequences. Unlike autoregressive models (e.g., LSTM, GRU), CSGN generates the entire sequence at once by transforming a sequence of latent vectors sampled from a Gaussian Process (GP). The model jointly models spatial and temporal structures using graph convolutions and a novel graph refining scheme.

2.1 Latent Vector Sequence Generation

The model begins by sampling a sequence of latent vectors $\mathbf{F}_0 = (\mathbf{z}_t)_{t=1}^{T_0}$ from a Gaussian Process:

$$\mathbf{z}_t^{(c)} \sim \mathcal{GP}(0, \kappa), \quad \kappa(t, t') = \exp\left(-\frac{|t - t'|^2}{2\sigma_c^2}\right)$$

where σ_c is the characteristic length-scale for channel c . A spectrum of σ_c values is used to model multiple temporal scales.

Sequence Generation via Graph Convolutions

The generator transforms the latent sequence $\mathbf{F}_0 \in \mathbb{R}^{C_0 \times 1 \times T_0}$ into a skeleton sequence $\mathbf{F} \in \mathbb{R}^{C \times V \times T}$ through a series of blocks:

$$\mathcal{F}_{k+1} = h(\text{Conv}_{st}(\text{Up}_{st}(\mathcal{F}_k)))$$

where:

- Up_{st} : Spatio-temporal upsampling operator
- Conv_{st} : Spatio-temporal graph convolution
- h : Non-linear activation (LeakyReLU)

2.2 Graph Upsampling and Pyramid

A graph pyramid is constructed by iteratively coarsening the skeleton graph. Upsampling is performed by embedding coarser graphs into finer ones and assigning values to new vertices via averaging.

Spatially-Varying Graph Convolution

Unlike standard graph convolution, spatially-varying convolution does *not* share weights across vertices, allowing better modeling of heterogeneous body parts.

2.3 Generative Adversarial Training

The generator G and discriminator D are trained adversarially using WGAN-GP. The discriminator uses:

$$\mathcal{F}_{k+1} = h(\text{Coar}_{st}(\text{Conv}_{st}(\mathcal{F}_k)))$$

where Coar_{st} performs spatio-temporal coarsening.

Action Manipulation via Conditional Sampling

CSGN supports conditional generation (e.g., prediction, completion) via an inverse mapping network H that maps observed skeletons to latent vectors. The conditional sampling process is:

$$p(\mathcal{F}_0|\mathcal{F}^K) \approx p(\mathcal{F}_0^U|\mathcal{F}_0^K) \cdot p(\mathcal{F}_0^K|\mathcal{F}^K)$$

where $\mathcal{F}^K$ is the observed part and $\mathcal{F}^U$ is the unobserved part.

Paper 2

A GAN-Like Approach for Physics-Based Imitation Learning and Interactive Character Control (2021)

Authors: Pei Xu, Ioannis Karamouzas

1 Dataset Used

The authors use the **LAFAN1 dataset** [?] for reference motion clips. The dataset contains motion capture recordings at 30Hz from multiple subjects, each 3–5 minutes long, covering a variety of behaviors. Short clips (1–3 seconds) are extracted and retargeted to a simulated humanoid character without manual post-processing.

2 Methodology

The paper introduces a GAN-inspired framework for training physics-based character controllers without explicit reward engineering, enabling real-time interactive control through policy switching. The method builds on **Generative Adversarial Imitation Learning (GAIL)**, which combines GANs with reinforcement learning. The key idea is to train a policy to imitate expert demonstrations by fooling a discriminator that distinguishes between expert and policy-generated trajectories.

2.1 Key Improvements Over GAIL

To stabilize training and improve performance, the authors introduce three modifications:

1. **Ensemble of Discriminators:** Multiple discriminators share feature extraction layers but have independent final layers. This prevents overfitting and reward vanishing.

2. **Hinge Loss:** Replaces binary cross-entropy with hinge loss:

$$\mathcal{L}_{D_i} = \mathbb{E}[\max(0, 1 + D_i(\mathbf{o}))] + \mathbb{E}[\max(0, 1 - D_i(\tilde{\mathbf{o}}))]$$

where $\mathbf{o}$ is the agent's observation and $\tilde{\mathbf{o}}$ is the expert's.

3. **Gradient Penalty:** Enforces 1-Lipschitz continuity via:

$$\mathcal{L}_{D_i}^{GP} = \mathbb{E} \left[(\|\nabla_{\hat{\mathbf{o}}} D_i(\hat{\mathbf{o}})\|_2 - 1)^2 \right]$$

where $\hat{\mathbf{o}}$ is an interpolation between expert and agent samples.

The total discriminator loss for N discriminators is:

$$\mathcal{L}_D = \frac{1}{N} \sum_{i=1}^N \left(\mathcal{L}_{D_i} + \lambda^{GP} \mathcal{L}_{D_i}^{GP} \right)$$

Observation and State Spaces

- **Discriminator Input:** 5 consecutive frames of local link positions and orientations relative to the root.
- **Policy Input:** 4 consecutive frames including link velocities.
- **Policy Output:** Target joint angles for PD controllers.

2.2 Policy Training

The policy is trained using **Distributed PPO (DPPO)** [?]. The reward is the clipped average score from the discriminator ensemble:

$$r_t = \frac{1}{N} \sum_{i=1}^N \text{clip}(D_i(\mathbf{o}_{t-3:t+1}), -1, 1)$$

Interactive Policy Switching

At runtime, a **policy switcher** uses the target policy’s discriminator score to decide when to switch:

$$\frac{1}{N} \sum_{i=1}^N \text{clip}(D_i^{\text{target}}(\mathbf{o}_{t-4:t}), -1, 1) \geq \tau$$

where τ is a threshold. This allows real-time response to user input without motion matching or phase variables.

Paper 3

GAN-based Reactive Motion Synthesis with Class-aware Discriminators for Human–Human Interaction (2021)

Authors: Qianhui Men, Hubert P. H. Shum, Edmond S. L. Ho, Howard Leung

1 Datasets Used

The authors evaluate their method on three datasets:

- **SBU Kinect Interaction Dataset (SBU)**: Contains 8 interaction types (e.g., *kick*, *push*, *hug*, *shake hands*) with 3D joint positions.
- **Human–Human–Object Interaction (HHOI)**: Includes interactions like *shake hands* and *high-five*, with longer sequences.
- **Character-Character Dataset (2C)**: A high-quality motion capture dataset with *kicking* and *punching* interactions.

2 Methodology

The key contributions include:

- A semi-supervised GAN system for reactive motion synthesis.
- A part-based attentive recurrent generator that models spatial-temporal dependencies.
- A dual discriminator with binary and multi-class classifiers to enhance motion realism and discriminability.

The system takes the active character’s motion as input and synthesizes the reactive character’s motion. The framework includes:

- A **generator** with part-based LSTM encoding and attentive decoding.

- A **discriminator** with binary and multi-class classifiers.
- Auxiliary **loss functions** to enforce physical and temporal constraints.

2.1 Generator: Part-based Attentive Recurrent Network

The generator encodes the active character's motion using a hierarchical LSTM that processes five body parts (four limbs and trunk) separately. The hidden states for each part are concatenated to form a global spatial representation.

The decoder uses an **attention mechanism** to align the reactive motion with the active motion. The context vector r_t at each decoder step is computed as:

$$r_t = \sum_{s=1}^S \alpha(s, t) h_s,$$

where $\alpha(s, t)$ is the attention weight:

$$\alpha(s, t) = \text{softmax}(V \tanh(W[h_s; \hat{h}_{t-1}])).$$

The LSTM gates in the decoder are updated as:

$$\begin{pmatrix} \hat{i}_t \\ \hat{f}_t \\ \hat{o}_t \\ \hat{u}_t \\ \hat{x}_t \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \\ \tanh \end{pmatrix} \hat{W} \begin{pmatrix} \hat{x}_{t-1}^B \\ \hat{h}_{t-1} \\ r_t \end{pmatrix}.$$

2.2 Discriminator: Class-aware Dual Classifier

The discriminator consists of:

- A **binary classifier** D_b to distinguish real vs. synthetic motions.
- A **multi-class classifier** D_m to predict the interaction type.

The two classifiers share most parameters, making the system efficient. Only the reactive motion is fed to the discriminator to force it to focus on reactive patterns.

2.3 Loss Functions

The total loss is a combination of adversarial and auxiliary losses:

Adversarial Loss

$$\begin{aligned}\mathcal{L}_{sup} &= -\mathbb{E}_{x,y \sim G} \log \frac{p_{D_m}(y|x, y < N+1)}{p_{D_m}(y|x, y = N+1)} + \mathbb{E}_{x,y \sim p_B} \log p_{D_m}(y|x, y < N+1) \\ \mathcal{L}_{unsup} &= \mathbb{E}_{x \sim p_B} \log D_b(x) + \mathbb{E}_{x \sim p_A} \log[1 - D_b(G(x))]\end{aligned}$$

Auxiliary Losses

- **Bone Loss:** Preserves bone lengths:

$$\mathcal{L}_{sll} = \sum_t \sum_j |skl(\hat{x}_t^B, j) - skl_{ref}(j)|$$

- **Continuity Loss:** Ensures smooth motion:

$$\mathcal{L}_{con} = \sum_t \max(\|\hat{x}_{t+\Delta t}^B - \hat{x}_t^B\|^2 - \|\hat{x}_{t+k\Delta t}^B - \hat{x}_t^B\|^2 + \lambda, 0)$$

- **Contractive Loss:** Reduces ambiguity:

$$\mathcal{L}_1 = \sum_t |\hat{x}_t^B - x_t^B|$$

Overall Objective

$$\min_G \max_D \mathcal{L}_{sup} + \mathcal{L}_{unsup} + \alpha \mathcal{L}_{sll} + \beta \mathcal{L}_{con} + \gamma \mathcal{L}_1$$

Paper 4

Generative Adversarial Graph Convolutional Networks for Human Action Synthesis (2022)

Authors: Bruno Degardin, Joao Neves, Vasco Lopes, etc

1 Datasets Used

The authors evaluate Kinetic-GAN on three publicly available datasets:

- **NTU RGB+D** – 56,880 videos, 60 action classes, 25 joints per skeleton.
- **NTU-120 RGB+D** – 114,480 videos, 120 action classes, 25 joints.
- **Human3.6M** – 2D human motions, 15 body joints, 10 action classes.

2 Methodology

The paper introduces **Kinetic-GAN**, a novel framework that combines Generative Adversarial Networks (GANs) with Graph Convolutional Networks (GCNs) to synthesize realistic and diverse human action sequences from skeleton data. The method addresses key limitations in existing approaches, such as limited action conditioning and poor long-term temporal modeling. Kinetic-GAN consists of a generator and a discriminator, both built using **spatiotemporal graph convolutional networks**. The generator synthesizes human action sequences from a latent vector $\mathbf{z}$ and an action class embedding $\mathbf{y}$, while the discriminator distinguishes between real and generated sequences.

2.1 Graph Convolutional Formulation

The skeleton is represented as a spatiotemporal graph $\mathcal{G}_l = (\mathcal{V}_l, \mathcal{E}_l)$ at resolution level l , with N_l joints and T_l frames. The spatial graph convolution is defined as:

$$\mathcal{S}(\mathbf{X}_l) = \sum_{i=1}^p \mathbf{A}_{l_i}^{-\frac{1}{2}} \mathbf{A}_{l_i} \mathbf{A}_{l_i}^{-\frac{1}{2}} \mathbf{X}_l \mathbf{W}_{l_i}$$

where:

- $\mathbf{X}_l \in \mathbb{R}^{N_l \times T_l \times C}$ is the feature map,
- $\mathbf{A}_{l_i}$ is the adjacency matrix for partition i ,
- $\mathbf{W}_{l_i}$ are learnable weights.

Temporal convolution is performed using 1D convolutions across frames.

Generator with Mapping Network

The generator uses a **non-linear mapping network** to transform the latent code $\mathbf{z}$ into an intermediate latent space $\mathcal{W}$, which improves disentanglement and linearity of variation factors. The upsampling process is defined as:

$$\mathbf{X}_{l+1} = \mathcal{T}(\mathcal{S}(Up(\mathbf{X}_l))) + \mathcal{T}(Up(\mathbf{X}_l))$$

where $\mathcal{T}$ and $\mathcal{S}$ are temporal and spatial graph convolutions, and Up is the upsampling operation.

2.2 Conditional Adversarial Training

The model is conditioned on action labels using embedded class representations $\mathbf{y}$. The objective function is based on **WGAN-GP**:

$$\min_G \max_D \mathbb{E}_{x \sim \mathbb{P}_r} [D(x|y)] - \mathbb{E}_{\tilde{x} \sim \mathbb{P}_g} [D(\tilde{x}|y)] + \lambda \mathbb{E}_{\hat{x} \sim \mathbb{P}_{\hat{x}}} [(\|\nabla_{\hat{x}} D(\hat{x}|y)\|_2 - 1)^2]$$

Stochastic Variation via Noise Injection

To increase diversity without affecting skeleton structure, Gaussian noise is injected at each layer:

$$\mathbf{X}_{l+1} = \mathcal{T}(\mathcal{S}(Up(\mathbf{X}_l))) + \mathcal{T}(Up(\mathbf{X}_l)) + \mathbf{r}_{l+1}\mathbf{w}_{l+1}$$

where $\mathbf{r}_{l+1}$ is random noise and $\mathbf{w}_{l+1}$ is a per-channel weight vector.

Truncation Trick

During inference, the latent vector $\mathbf{w}$ is scaled toward the center of $\mathcal{W}$ to improve quality:

$$\mathbf{w}' = \mathbb{E}_{\mathbf{z} \sim \mathbb{P}_{\mathbf{z}}}[f(\mathbf{z})] + \psi(\mathbf{w} - \mathbb{E}_{\mathbf{z} \sim \mathbb{P}_{\mathbf{z}}}[f(\mathbf{z})])$$

where $\psi \leq 1$ controls the trade-off between quality and diversity.

Paper 5

Composite Motion Learning with Task Control (2023)

Authors: Pei Xu, Xiumin Shang, Victor Zordan, Ioannis Karamouzas

1 Dataset Used

- **LAFAN1 dataset** (Harvey et al., 2020)
- Other commercial and publicly available motion capture datasets
- Motion data recorded at 30Hz, including:
 - Short clips (1–3 seconds) for single-clip imitation
 - Collections of motions (15–70 seconds) for goal-directed tasks
 - Specific motions: crouch, walk, run, tennis footwork, tennis swing, aiming, juggling

2 Methodology

The paper proposes a novel framework for physics-based character control that combines composite motion learning and task-driven objectives using deep reinforcement learning.

2.1 Core Components

Full-Body Motion Decoupling

- Character pose represented as $\mathcal{P}_t := \{(\rho_l, q_l, \dot{\rho}_l, \dot{q}_l)\}_{l=1}^{N_{\text{link}}}$
- Body divided into partial-body groups (e.g., upper/lower body)
- Each group i has its own reference motion collection $\mathcal{M}^i$
- Observation space for discriminator: $o_t^i := \{\mathcal{P}_{t-n_i}^i, \dots, \mathcal{P}_t^i, \mathcal{P}_{t+1}^i\}$

GAN-like Imitation Learning

- Multiple discriminator ensembles $D^i = \{D_n^i\}_{n=1}^N$
- Loss function with hinge loss and gradient penalty:

$$\mathcal{L}_{D^i} = \frac{1}{N} \sum_{n=1}^N \left(\mathbb{E}_t [\max(0, 1 + D_n^i(o_t^i))] + \mathbb{E}_t [\max(0, 1 - D_n^i(\tilde{o}_t^i))] + \lambda^{\text{GP}} \mathbb{E}_t [(\|\nabla_{\tilde{o}_t^i} D_n^i(\tilde{o}_t^i)\|_2 - 1)^2] \right)$$

- Imitation reward: $r_t^{D^i} = \frac{1}{N} \sum_{n=1}^N \text{Clip}(D_n^i(o_t^i), -1, 1)$

Multi-Objective Learning Framework

- Policy optimization: $\mathcal{L}_\pi = \sum_{k=1}^K \mathbb{E}_t [\omega_k \bar{A}_t^k \log \pi(\mathbf{a}_t | \mathbf{s}_t, \mathbf{g}_t)]$
- Standardized advantage: $\bar{A}_t^k = \frac{A_t^k - \mathbb{E}_t[A_t^k]}{\sqrt{\text{Var}_t[A_t^k]}}$
- Computationally efficient form:

$$\mathcal{L}_\pi = \mathbb{E}_t \left[\left(\sum_k \omega_k \bar{A}_t^k \right) \log \pi(\mathbf{a}_t | \mathbf{s}_t, \mathbf{g}_t) \right]$$

- **Multi-critic architecture** with **PopArt** normalization for value targets

Incremental Learning

- Cooperative policy: $\pi(\mathbf{a}_t | \mathbf{s}_t, \mathbf{g}_t, \mathbf{a}_t^{\text{meta}}) := \mathcal{N}(\mu_t + \mathbf{w}_t \text{Stop}(\mathbf{a}_t^{\text{meta}}), \sigma_t^2)$
- Weight vector $\mathbf{w}_t$ controls reliance on meta-policy
- Enables sample-efficient training by reusing pre-trained policies

2.2 Theoretical Backing

- **Policy Gradient Theorem:** Foundation for PPO-based optimization
- **Generative Adversarial Networks:** Multiple discriminators enable composite motion learning without manual reward shaping
- **Multi-Objective Optimization:** Avoids scalarization of rewards through independent advantage estimation
- **Linearity of Expectation:** Enables efficient combined policy update (Eq. 7 $\rightarrow$ Eq. 9)

Paper 6

ActFormer: A GAN-based Transformer towards General Action-Conditioned 3D Human Motion Generation (2023)

Authors: Liang Xu, Ziyang Song, etc

1 Datasets Used

The paper utilizes several datasets for training and evaluation:

- **NTU-13:** Subset of NTU RGB+D 120 with 13 action categories.
- **NTU RGB+D 120:** Large-scale dataset with 120 action categories (94 single-person, 26 two-person).
- **NTURecon-1P:** SMPL-based version of NTU-1P reconstructed using a motion reconstruction pipeline.
- **BABEL:** Large-scale, high-quality SMPL-based motion dataset with fine-grained action labels.
- **GTA Combat:** New synthetic dataset introduced in this paper, containing complex multi-person combat behaviors (2–5 persons) collected from Grand Theft Auto V.

2 Methodology

The paper introduces a novel framework for generating 3D human motions conditioned on semantic action labels, supporting both single-person and multi-person interactive actions. ActFormer is a GAN-based Transformer framework for action-conditioned 3D human motion generation. It uses:

- A **Gaussian Process (GP) latent prior** to model temporal correlations.
- A **Transformer-based generator** to model spatio-temporal dependencies.

- A **GAN training scheme** for high-quality generative modeling.
- **Alternating Transformer encoders** for multi-person interactions.

2.1 Single-Person Motion Generation

- **Latent Prior:** A latent vector sequence $z \in \mathbb{R}^{T \times C_0}$ is sampled from a Gaussian Process (GP) prior. Each channel is an independent 1D GP with varying length-scales to capture multi-scale temporal correlations.
- **Generator:** The Transformer generator takes z and an action label a . The input tokens are:
 - T tokens from z (after MLP embedding)
 - One class token for action a
- **Positional Encoding:** Learnable positional encodings (PE) are added to tokens.
- **Temporal Transformer (T-Former):** Models temporal correlations across frames.
- **Output:** The class token is discarded; remaining T tokens are projected to motion parameters (root translation l_t and body poses θ_t).

2.2 Multi-Person Motion Generation

- **Input:** The same latent sequence z is shared across P persons to enforce synchronization.
- **Positional Encoding:** 2D PE is used: $PE(t, p) = \text{concat}(TPE(t), PPE(p))$, where TPE is temporal and PPE is person-wise PE.
- **Alternating Transformers:**
 - **I-Former:** Models interactions among persons at each time step.
 - **T-Former:** Models temporal correlations for each person independently.
- **Output:** $P \cdot T$ tokens are projected to (P, T, C) motion sequences.

2.3 Generative Adversarial Training

- **Loss:** Conditional Wasserstein GAN loss:

$$L_D = \mathbb{E}[D(G(z, a), a) - D(\tilde{M}, a)],$$

$$L_G = \mathbb{E}[-D(G(z, a), a)],$$

where $\tilde{M}$ is real motion data.

- **Discriminator:** ST-GCN is used. Multi-person motions are concatenated along the feature dimension. **Permutation augmentation** is applied during training to handle person order invariance.

Paper 7

Policy-Space Diffusion for Physics-Based Character Animation (2025)

Authors: Michelle Rocca, Sune Darkner, Kenny Erleben, Sheldon Andrews

1 Dataset Used

The authors used two primary datasets:

- **GroundTruth-set:** Derived from the LaFan1 dataset by Harvey et al. (2020). It consists of motion capture data segmented into shorter clips representing individual motion types.
- **Policy-set:** A custom dataset of control policies trained using imitation learning on the GroundTruth-set. It includes policies for various motions, morphologies, and terrains.

2 Methodology

The core idea is to treat control policy weights as points on a differentiable manifold. The authors hypothesize that policies for similar motions should lie close in this manifold. This enables generative modeling of policies using diffusion models.

2.1 Mathematical Formulation

Policy Representation

A policy $P_{\mathbf{w}}^{\mathbf{m}}$ with weights $\mathbf{w}$ is trained to imitate a motion clip $\mathbf{m}$. The goal is to ensure that policies for similar motions have similar weights:

$$\mathbf{w}' \approx \mathbf{w}'' \quad \text{if} \quad \mathbf{m}' \sim \mathbf{m}''$$

Common Neighbor Policy (CNP) Regularization

To enforce weight similarity, a regularization term is added during policy training:

$$\mathcal{L} = \mathcal{L}_{\text{AMP}} + \lambda \|\mathbf{w} - \mathbf{w}_{\text{CNP}}\|^2$$

where $\mathbf{w}_{\text{CNP}}$ is a pre-trained policy weight vector, and $\lambda = 0.02$ controls the regularization strength.

Diffusion Model for Policy Generation

A conditional Diffusion Transformer is used to generate policy weights. The model is trained to denoise Gaussian noise into policy parameters. The conditioning is applied by concatenating condition vectors (e.g., morphology parameters) with the input at each diffusion step.

Policy Architecture

The policy network is a compact MLP with:

- Input: 223-dimensional state
- Output: 28-dimensional action (mean of Gaussian distribution)
- Hidden layers: 128 and 32 neurons
- Total parameters: 33k (reduced from 770k in original AMP)

Conditioning Strategy

- Categorical conditions (e.g., motion type) are encoded using PCA-derived continuous vectors.
- Continuous conditions (e.g., height, terrain roughness) are min-max normalized.
- The first 12 PCA components are used, capturing >80% of variance.

Evaluation Metrics

- **Falling Rate (FR):** Percentage of simulations where the character falls.

- **Dynamic Time Warping (DTW):** Measures motion similarity between simulated and ground-truth trajectories.
- **Fréchet Motion Distance (FMD):** Compares distributions of motion features in a latent space.

Paper 8

Policy-space Interpolation for Physics-based Characters (2025)

Authors: Michele Rocca, Sheldon Andrews, Kenny Erleben

1 Introduction

2 Dataset Used

The paper does not explicitly name a public dataset but relies on **motion capture (MoCap) data** for training imitation policies. The policies are trained using the **Adversarial Motion Prior (AMP)** framework, which uses a discriminator to encourage motion realism. The policies are trained for various skills such as Walk, Run, Crouch, Crawl, Idle, Roll, Get-up, and multiple dance styles.

3 Methodology

The work introduces a novel method for blending control policies for physics-based characters by performing linear interpolation directly in the weight space of neural network policies, rather than blending their outputs. This approach reduces computational cost while enabling smooth motion transitions and variation synthesis without retraining.

The core idea is to interpolate between pre-trained policy networks in **weight space** rather than **action space**. This is motivated by the observation that policies with similar behaviors have similar network weights when trained with a regularization strategy.

Policy Regularization

The authors extend the **Common Neighbor Policy (CNP)** regularization strategy to a **graph-based structure**. The loss function for training a policy is:

$$\mathcal{L} = c_{\text{Disc}}\mathcal{L}_{\text{Disc}} + c_{\text{Task}}\mathcal{L}_{\text{Task}} + \gamma\|\mathbf{w} - \mathbf{w}_{\text{Reg}}\|^2$$

where:

- $\mathbf{w}_{\text{Reg}}$: weights of a pre-trained policy used as a regularizer
- $c_{\text{Disc}}, c_{\text{Task}}$: weights for imitation and task losses
- γ : regularization strength

Graph-Based Regularization

Policies are organized in a graph where edges represent regularization directions (see Fig. 2 in the paper). For example:

- Walk $\rightarrow$ Sneaky $\rightarrow$ Jogging $\rightarrow$ Running
- Idle $\rightarrow$ Dance1, Dance2, ...

Linear Interpolation in Weight Space

Given a path between policies π_0 and π_n via intermediate policies $\pi_1, \dots, \pi_{n-1}$, the interpolated policy at parameter $s \in [0, 1]$ is:

$$\pi(s) = (1 - s')\pi_k + s'\pi_{k+1}$$

where $s' = ns - k$ for $k = \lfloor ns \rfloor$.

For multiple policies, barycentric coordinates are used:

$$\pi = \lambda_1\pi_1 + \lambda_2\pi_2 + \dots + \lambda_m\pi_m$$

with $\lambda_i \geq 0$ and $\sum \lambda_i = 1$.

Sampling Strategy

To test interpolation, coefficients λ_i are sampled with rejection to ensure:

- One policy has $\lambda > 0.80$, or

- Two policies have $\lambda > 0.40$, or
- Three policies have $\lambda > 0.25$

Evaluation Metrics

- **Fréchet Motion Distance (FMD):** Measures motion similarity using latent space encodings of pose sequences.
- **Success/Failure Criteria:** Based on motion type (e.g., falling for standing motions, distance traveled for crawling).

3.1 Network Architecture and Training

- **Policy Network:** MLP with two hidden layers [128, 32], ReLU activations.
- **Input:** 225-D state (root height, joint positions, velocities, target location).
- **Output:** 28-D action vector (mean of Gaussian).
- **Training Phases:**
 1. Imitation-only ($c_{\text{Disc}} = 1$)
 2. Task learning ($c_{\text{Task}} > 0$)
 3. Fine-tuning on rough terrain

Paper 9

Learning 3D Human Dynamics from Video (2019)

Authors: Angjoo Kanazawa, Jason Y. Zhang, Panna Felsen, Jitendra Malik

1 Datasets Used

The paper utilizes a multi-tiered dataset strategy to train and evaluate the proposed model, HMMR (Human Mesh and Motion Recovery):

- **Human3.6M:** Large-scale motion capture dataset with 3D ground truth annotations. Used for training and validation.
- **Penn Action:** In-the-wild video dataset with 2D pose annotations for sports actions.
- **NBA (Ours):** Custom dataset of basketball players shooting 3-pointers, with 2D pose annotations.
- **VLOG-people:** Subset of the VLOG lifestyle dataset, filtered for consistent OpenPose detections. Provides pseudo-ground truth 2D poses.
- **InstaVariety:** Large-scale dataset collected from Instagram using activity-specific hash-tags. Provides pseudo-ground truth 2D poses via OpenPose.
- **3DPW (3D Poses in the Wild):** Used exclusively for testing; contains in-the-wild videos with 3D ground truth from IMUs.

2 Methodology

The proposed framework, HMMR, learns a temporal representation of 3D human dynamics from video and transfers it to static images. The methodology consists of the following components:

2.1 Problem Formulation

Given a video $V = \{I_t\}_{t=1}^T$, the goal is to learn a temporal encoder f_{movie} that maps a window of image features $\{\phi_t\}$ to a dynamics-aware representation Φ_t . A hallucinator h is also learned to map a single image feature ϕ_t to $\tilde{\Phi}_t$, enabling motion prediction from static images.

2.2 3D Human Mesh Representation

The 3D human body is represented using the SMPL model:

$$\mathcal{M}(\beta, \theta) \in \mathbb{R}^{N \times 3}, \quad N = 6890$$

where:

- $\beta \in \mathbb{R}^{10}$: shape parameters
- $\theta \in \mathbb{R}^{72}$: pose parameters (global rotation + 23 joints)
- $\Pi = [s, t_x, t_y]$: weak-perspective camera parameters

3D joints are regressed as $X = W\mathcal{M}(\beta, \theta)$, and projected to 2D as $x = \Pi(X)$.

2.3 Temporal Encoder and 3D Regression

The temporal encoder f_{movie} is a 1D fully convolutional network (ResNet-based) that processes a window of 13 frames. It outputs Φ_t , which is fed into:

- A 3D regressor f_{3D} : Predicts $\Theta_t = \{\beta, \theta, \Pi\}$
- Delta predictors $f_{\Delta t}$: Predict pose changes $\Delta\theta$ for $t \pm \Delta t$

2.4 Loss Functions

The model is trained with a combination of losses:

$$\begin{aligned} L_{2D} &= \|v_t(x_t - \hat{x}_t)\|_2^2 \\ L_{3D} &= \|\Theta_t - \hat{\Theta}_t\|_2^2 \quad (\text{if available}) \\ L_{\text{adv prior}} &= \sum_k (D_k(\Theta) - 1)^2 \\ L_{\beta \text{ prior}} &= \text{Shape prior} \\ L_{\text{const shape}} &= \sum_{t=1}^{T-1} \|\beta_t - \beta_{t+1}\| \\ L_{\text{hal}} &= \|\Phi_t - \tilde{\Phi}_t\|_2 \end{aligned}$$

The overall objective is:

$$L = L_{\text{temporal}} + L_{\text{hal}} + L_t(\tilde{\Phi}_t) + \sum_{\Delta t} L_{t+\Delta t}(\tilde{\Phi}_t)$$

2.5 Training Strategy

The model is trained in a semi-supervised manner using:

- Fully supervised data (Human3.6M)
- 2D-annotated in-the-wild data (Penn Action, NBA)
- Pseudo-ground truth 2D poses from unlabeled videos (VLOG, InstaVariety)

Paper 10

Predicting 3D Human Dynamics from Video (2019)

Authors: Jason Y. Zhang, Panna Felsen, Angjoo Kanazawa, Jitendra Malik

1 Datasets Used

The authors used the following datasets for training and evaluation:

- **Human3.6M** [?]: A large-scale motion capture dataset with 3D ground truth annotations. Used Subjects 1, 6, 7, 8 for training; Subject 5 for validation; and Subjects 9 and 11 for testing.
- **Penn Action** [?]: In-the-wild sports videos with 2D keypoint annotations for 15 activities (e.g., bowling, golf swing).
- **NBA** [?]: Videos of professional basketball players attempting 3-point shots, with 2D keypoint annotations.
- **InstaVariety** [?]: Large-scale Internet videos from Instagram with pseudo-ground truth 2D keypoints from OpenPose [?].

2 Methodology

The paper introduces **Predicting Human Dynamics (PHD)**, an autoregressive framework for predicting future 3D human mesh sequences from past video frames. The approach consists of three main components:

2.1 3D Mesh Representation

The human body is represented using the **SMPL model** [?], a differentiable function $\mathcal{M}(\beta, \theta)$ that outputs a 3D mesh with 6890 vertices. The parameters are:

- Pose parameters $\theta \in \mathbb{R}^{72}$ (axis-angle representation for 23 joints + global rotation)
- Shape parameters $\beta \in \mathbb{R}^{10}$ (PCA coefficients)

A weak-perspective camera model $\Pi = [s, t_x, t_y]$ is used for 2D projection.

2.2 Causal Temporal Encoder

A causal convolutional network f_{movie} encodes past image features ϕ_t (from ResNet-50) into a latent "movie strip" Φ_t representing motion history:

$$\Phi_t = f_{\text{movie}}(\phi_{t-r}, \dots, \phi_t)$$

A 3D regressor f_{3D} maps Φ_t to SMPL parameters $\Theta_t = [\theta_t, \beta_t]$ and camera Π_t :

$$\Theta_t, \Pi_t = f_{3D}(\Phi_t)$$

2.3 Autoregressive Prediction

An autoregressive model f_{AR} predicts future latent representations:

$$\tilde{\Phi}_t = f_{AR}(\Phi_1, \dots, \Phi_{t-1})$$

The joint distribution of future sequences is modeled as:

$$p(\Phi) = \prod_{t=1}^T p(\Phi_t | \Phi_1, \dots, \Phi_{t-1})$$

The predicted meshes are decoded via $f_{3D}(\tilde{\Phi}_t)$.

2.4 Loss Functions

The model is trained with a combination of losses:

- **3D Loss:** $L_{3D} = \|X_t - \tilde{X}_t\|_2^2 + \|\theta_t - \tilde{\theta}_t\|_2^2 + \|\beta - \hat{\beta}\|_2^2$
- **2D Reprojection Loss:** $L_{2D} = \|v_t \odot (x_t - \hat{x}_t)\|_2^2$

- **Adversarial Prior Loss:** $L_{\text{adv prior}}$ from [?]
- **Shape Prior:** $L_{\text{beta loss}}$ from [?]
- **Consistency Loss:** $L_{\text{const}} = \sum_{t=1}^{T-1} \|\beta_t - \beta_{t+1}\|_2^2$
- **Distillation Loss:** $L_{\text{movie strip}} = \|\Phi_t - \tilde{\Phi}_t\|$

Total loss for autoregressive training:

$$L_{\text{AR}} = \sum_t L_t + L_{\text{const}} + \sum_t L_{\text{movie strip}}$$

Paper 11

Novel View Synthesis of Human Interactions from Sparse Multi-view Videos (2022)

Authors: Qing Shuai, Chen Geng, Qi Fang, Sida Peng, Wenhao Shen, Xiaowei Zhou, Hujun Bao

1 Datasets Used

- **Multi-Human Dataset:** A custom dataset with 4 dynamic scenes involving human interactions, captured by 22 synchronized cameras. Eight cameras were used for training and four for testing.
- **ST-NeRF Dataset:** Provided by Zhang et al. (2021), containing sequences with 16 cameras covering up to 180 degrees.
- **Soccer Dataset:** A real-world soccer game captured using 8 static GoPro cameras at 60 fps, used to evaluate in-the-wild performance.

2 Methodology

The proposed system reconstructs a layered neural representation of dynamic multi-person scenes from sparse multi-view videos. The pipeline consists of three main stages:

2.1 Multi-entity Motion Capture

- **Camera Calibration & Synchronization:** Cameras are calibrated and synchronized manually. A pose-guided synchronization method is introduced to compensate for temporal misalignment by minimizing 2D reprojection loss:

$$\min_{[\Delta_v]} \sum_{v=1}^V \mathcal{L}_{2d}(\theta + \Delta t_v \hat{\theta}, \beta, R + \Delta t_v \hat{R}, T + \Delta t_v \hat{T}, W_v, w_v)$$

- **Human Pose Detection & Matching:** 2D human poses and object bounding boxes are detected and matched across views. 3D keypoints are triangulated.
- **Parametric Model Fitting:** SMPL+H models are fitted to 3D keypoints using the loss:

$$\mathcal{L}_{\text{mcap}} = \mathcal{L}_{3\text{d}} + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}} + \lambda_t \mathcal{L}_t$$

where $\mathcal{L}_{3\text{d}}$ is the 3D keypoint distance, $\mathcal{L}_{\text{reg}}$ regularizes SMPL parameters, and $\mathcal{L}_t$ enforces temporal smoothness.

2.2 Layered Neural Scene Representation

The scene is represented as multiple neural radiance fields (NeRF) for each human, object, and background.

- **Human Representation:** Uses NeuralBody’s approach with latent codes anchored on SMPL vertices.
- **Object Representation:** Balls are modeled as radiance fields with constant densities and time-varying colors. The transformation to canonical space is:

$$o^m(p_{tt}), c^m(p_{tt}, d) = \delta^m(p_{tt} - \mathbf{T}_t^m), \epsilon^m(p_{tt} - \mathbf{T}_t^m, d, \mathbf{l}_t)$$

- **Background Representation:** A NeRF with temporal latent codes, limited to a region around the ground plane.
- **Rendering:** Layered volume rendering is performed. The color of a ray $\mathbf{r}$ is computed as:

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N o(p_i) \prod_{j < i} (1 - o(p_j)) c(p_i, \mathbf{r})$$

2.3 Network Training

The model is trained with:

- **Rendering Loss:**

$$\mathcal{L}_{\text{rgb}} = \sum_{\mathbf{r} \in \mathcal{R}} \|\hat{C}(\mathbf{r}) - C(\mathbf{r})\|_2^2$$

- **Sparsity Regularization:** An entropy loss encourages each ray to be assigned to one layer:

$$\mathcal{L}_{\text{ent}} = - \sum_{\mathbf{r} \in \mathcal{R}} \sum_{m=1}^{|\alpha(\mathbf{r})|} \alpha_m \log(\alpha_m)$$

- **Keypoint Supervision:** Uses detected 2D keypoints to guide layer assignment:

$$\mathcal{L}_{\text{kpt}} = - \sum_{\mathbf{r} \in \mathcal{R}} w(\mathbf{r}) \|\alpha(\mathbf{r}) - \hat{\alpha}(\mathbf{r})\|$$

Paper 12

ViMo: Generating Motions from Casual Videos (2024)

Authors: Liangdong Qiu, Chengxing Yu, Yanran Li, Zhao Wang, Haibin Huang, Chongyang Ma, Di Zhang, Pengfei Wan, Xiaoguang Han

1 Datasets Used

The paper utilizes several datasets for training and evaluation:

1.1 Primary Training Dataset

- **AIST++:** Contains 1,408 high-quality dance motions with music from 10 genres, with 3D motions and 2D videos from 8 camera angles.
- **Training Split:** 8 dance genres for training, **break** and **middle hip-hop** for testing.

1.2 Generated Dataset

- **Chinese Classic Dance Dataset:**
 - 52 solo-dance videos from Chinese annual dancing competition
 - Processed with AlphaPose for 2D pose extraction
 - Generated 750 3D motion clips (63 minutes total) using ViMo
 - Used for music-to-motion generation and style transfer tasks

1.3 Other Referenced Datasets

- Human3.6M
- AMASS
- KIT Motion-Language
- Finedance

2 Methodology

ViMo addresses the novel problem of **video-to-motion generation** from *casual videos* - videos characterized by complex camera movements, montages, occlusions, and varying perspectives. Given an input video $\mathcal{V}$ containing a single person performing actions, the goal is to generate multiple plausible 3D human motions $\mathbf{M}$ that are semantically aligned with the video content in terms of pose shape and rhythm, without requiring explicit camera parameter estimation.

2.1 Technical Framework Overview

Input Processing Pipeline

- **2D Pose Extraction:** Using pose estimators like AlphaPose or OpenPose to extract 2D skeleton sequences
- **Input Representation:** $p \in \mathbb{R}^{S \times J_{2d} \times C_{in}}$ where:
 - $S = 150$ (sequence length, 5 seconds at 30 FPS)
 - $J_{2d} = 17$ (COCO format joints)
 - $C_{in} = 3$ (x, y coordinates + confidence score)
- **Preprocessing:** Missing/low-confidence joints are filtered, interpolation applied for continuity

Output Motion Representation

The target 3D motion $\mathbf{M}$ is represented per frame as:

$$\mathbf{m} \in \mathbb{R}^{S \times (24 \times 6 + 4 + 3)} \quad (12.1)$$

Breaking down the 151-dimensional vector per frame:

- **Joint Rotations (144D):** 24 joints in 6D rotation representation
- **Foot Contact Labels (4D):** Binary indicators for [left foot, right foot, left toe, right toe]
- **Root Position (3D):** Global (x, y, z) translation of pelvis

2.2 Diffusion Model Foundation

Forward Noising Process

The diffusion process follows a Markov chain, progressively adding Gaussian noise:

$$q\left(m_t^{1:S} \mid m_{t-1}^{1:S}\right) = \mathcal{N}\left(\sqrt{\alpha_t} m_{t-1}^{1:S}, (1 - \alpha_t) I\right) \quad (12.2)$$

The closed-form expression for any step t given clean data m_0 is:

$$m_t = \sqrt{\bar{\alpha}_t} m_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \quad (12.3)$$

where $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.

Reverse Denoising Process

The denoising network directly predicts the clean motion:

$$\hat{m}_0 = D(m_t, t, p) \quad (12.4)$$

The training objective uses the simple loss:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{m \sim q(\mathbf{M}|p), t \sim [1, N]} [\|m_0 - D(m_t, t, p)\|_2^2] \quad (12.5)$$

Analytical Posterior Distribution

The true posterior $q(m_{t-1} \mid m_t, m_0)$ is Gaussian with known parameters:

$$\hat{\mu}_t(m_t, m_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} m_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} m_t \quad (12.6)$$

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t, \quad \text{where } \beta_t = 1 - \alpha_t \quad (12.7)$$

2.3 Network Architecture Details

Denoiser Network $D(m_t, t, p)$

- **Input:** Noisy motion m_t , diffusion timestep t , 2D pose sequence p
- **Architecture:** Transformer-based with three denoising modules
- **Components:**
 - Self-attention blocks for temporal coherence
 - FiLM conditioning blocks for feature-wise modulation
 - Cross-attention and MLP layers

FiLM Conditioning Mechanism

Feature-wise Linear Modulation dynamically scales and shifts features based on conditioning:

$$\mathbf{h}' = \gamma(\mathbf{c}) \odot \mathbf{h} + \beta(\mathbf{c}) \quad (12.8)$$

where:

- $\mathbf{h}$: Intermediate features of shape $n \times d$ (n frames, d hidden dimension)
- $\mathbf{c} = [\text{embed}(t); \text{encode}(p)]$: Concatenated timestep and pose embeddings
- γ, β : Functions mapping condition $\mathbf{c}$ to modulation parameters

Classifier-Free Guidance

During training, conditioning is randomly dropped 25% of the time. During sampling:

$$\hat{m}_0 = (1 + w)D(m_t, t, p) - wD(m_t, t, \phi) \quad (12.9)$$

where w controls guidance strength, balancing fidelity and diversity.

2.4 Motion Representation Details

6D Rotation Representation

Instead of problematic Euler angles or constrained quaternions, ViMo uses the continuous 6D representation:

$$\mathbf{R} = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_1 \times \mathbf{a}_2] \quad (12.10)$$

where $\mathbf{a}_1, \mathbf{a}_2$ are the first two columns of the rotation matrix. This ensures:

- Continuity for neural network learning
- No gimbal lock issues
- Easy reconstruction of orthonormal rotation matrix

Foot Contact Label Computation

Foot contact labels are computed self-supervised from foot velocities:

$$f_i = \mathbb{I}(\|\mathbf{V}_{\text{foot}}\|_2 < \varepsilon) \quad (12.11)$$

where $\mathbf{V}_{\text{foot}}$ is the velocity of foot joints and ε is a threshold.

2.5 Loss Functions Formulation

The complete training objective combines diffusion and auxiliary losses:

$$\mathcal{L} = \mathcal{L}_{\text{simple}} + \lambda_1 \mathcal{L}_{\text{joints}} + \lambda_2 \mathcal{L}_{\text{vel}} + \lambda_3 \mathcal{L}_{\text{foot}} \quad (12.12)$$

Joints Loss (3D Position Error)

$$\mathcal{L}_{\text{joints}} = \frac{1}{S} \sum_{i=1}^S \|FK(R_0^i) - FK(\hat{R}_0^i)\|_2^2 \quad (12.13)$$

The Forward Kinematics function $FK(\cdot)$ converts joint rotations to 3D positions:

$$\mathbf{p}_j = \mathbf{p}_{par(j)} + \mathbf{R}_{par(j)} \cdot \mathbf{b}_j \quad (12.14)$$

where $\mathbf{b}_j$ is the bone vector in parent’s local coordinates.

Velocity Loss (Temporal Smoothness)

$$\mathcal{L}_{vel} = \frac{1}{S-1} \sum_{i=1}^{S-1} \left\| (R_0^{i+1} - R_0^i) - (\hat{R}_0^{i+1} - \hat{R}_0^i) \right\|_2^2 \quad (12.15)$$

This ensures correct angular velocity and eliminates jitter by enforcing smooth transitions.

Foot Contact Loss (Physical Plausibility)

$$\mathcal{L}_{foot} = \frac{1}{S-1} \sum_{i=1}^{S-1} \left\| (FK(\hat{R}_0^{i+1}) - FK(\hat{R}_0^i)) \cdot \hat{f}_i \right\|_2^2 \quad (12.16)$$

Key aspects:

- Uses model’s own predicted contact labels $\hat{f}_i$
- Masks velocities of joints predicted to be in contact
- Creates self-supervised learning of foot-ground interaction

The masking operation:

$$\mathbf{V}_{masked}^i = \mathbf{m}_i \odot \mathbf{V}^i \quad (12.17)$$

where $\mathbf{m}_i$ is the contact mask broadcast across spatial dimensions.

2.6 Training and Sampling Procedure

Training Details

- **Sequence Length:** 150 frames (5 seconds at 30 FPS)
- **Diffusion Steps:** $T = 1000$
- **Optimizer:** ADAN with learning rate 1×10^{-4}

- **Sampling Acceleration:** DDIM with 50 steps
- **Conditioning:** Multi-view 2D poses from different camera angles

Sampling Process

At each denoising step:

1. Network predicts $\hat{m}_0 = D(m_t, t, p)$
2. Compute posterior mean using analytical formula:

$$\hat{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\hat{m}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}m_t \quad (12.18)$$

3. Sample $m_{t-1} \sim \mathcal{N}(\hat{\mu}_t, \tilde{\beta}_t I)$
4. Repeat until $t = 0$ for final motion $\hat{m}_0$

2.7 Advanced Applications Implementation

Zero-Convolution for Few-Shot Stylization

For adapting pre-trained models to new styles without catastrophic forgetting:

$$W_{\text{effective}} = W_{\text{pretrained}} + W_{\text{new}} \quad (12.19)$$

where W_{new} is initialized to zeros, ensuring:

- Initial behavior identical to pre-trained model
- Gradual adaptation to target style
- Preservation of music-dance alignment knowledge

Architecture uses residual connection:

$$\text{Output} = f_{\text{frozen}}(\text{input}) + f_{\text{zero-conv}}(\text{input}) \quad (12.20)$$

Video-Guided Motion Completion

For motion inpainting and editing tasks:

$$\hat{\mathbf{m}}_{t-1} = \mathbf{b} \odot \mathbf{q}(c_{\text{const}}, t-1) + (1 - \mathbf{b}) \odot \hat{\mathbf{m}}_{t-1} \quad (12.21)$$

where:

- $\mathbf{b}$: Binary mask indicating known regions
- c_{const} : Constraint motion provided by user
- Enables style transfer and missing motion completion

2.8 Key Technical Innovations

Robustness to Camera Complexity

- No explicit camera parameter estimation required
- Multi-view 2D pose conditioning provides view invariance
- Diffusion process handles ambiguity in 2D-to-3D lifting

Self-Supervised Physical Constraints

- Foot contact labels learned jointly with motion generation
- No manual annotation or fixed thresholds required
- Model discovers physically plausible contact patterns

Structured Motion Representation

- Combined rotation, translation, and contact information
- 6D rotation ensures continuity and avoids singularities
- Hierarchical loss functions for different motion aspects

This comprehensive methodology enables ViMo to handle the challenges of casual videos while generating diverse, physically plausible 3D motions that maintain semantic alignment with input video content.

2.9 Applications

Dataset Construction

ViMo generates large-scale motion datasets from video collections, demonstrated with Chinese classic dance dataset (750 motions from 52 videos).

Few-Shot Dancing Stylization

- Uses **zero-convolution blocks** for parameter-efficient fine-tuning
- Adapts pre-trained EDGE model to new styles
- Mathematical formulation: $W_{\text{effective}} = W_{\text{pretrained}} + W_{\text{new}}$ with W_{new} initialized to zeros

Video-Guided Motion Completion

Implements motion inpainting using masked denoising:

$$\hat{m}_{t-1} = b \odot q(c_{\text{const}}, t-1) + (1-b) \odot \hat{m}_{t-1} \quad (12.22)$$

where b is a binary mask for known regions.